

BML-WAF: A Framework Based on Blockchain and Machine Learning for Web Application Firewall Log Integrity and Analysis

1st Ningyang Tang

*International School
Beijing University of
Posts and Telecommunications
Beijing, China
2023213293@bupt.cn*

2nd Yuxing Duan

*Beijing Advanced Innovation Center for
Future Blockchain and Privacy Computing
Beijing, China
duanyuxing@icloud.com*

3rd Qingsong Li

*Beijing Advanced Innovation Center for
Future Blockchain and Privacy Computing
Beijing, China
1320918493@qq.com*

4th Hang Cui

*School of Cyberspace Security
Beijing University of
Posts and Telecommunications
Beijing, China
ch1001@bupt.edu.cn*

Abstract—To address the dual challenges of susceptibility to tampering and high false positive rates in Web Application Firewall (WAF) logs, this paper proposes BML-WAF(Blockchain Machine Learning-based WAF), a novel framework that integrates blockchain and machine learning. For secure storage, a hybrid on-chain/off-chain architecture is introduced to guarantee log integrity by immutably anchoring cryptographic hashes on the blockchain. For accurate analysis, the framework employs a multi-stage pipeline featuring semantic extraction, rule-based calibration, and heterogeneous feature fusion to precisely discern real attacks. Extensive experiments on a highly imbalanced real-world dataset demonstrate its exceptional performance: the framework achieves 99.8% tamper detection accuracy and reduces the false positive rate by 77.27% compared to a Random Forest baseline, while also exhibiting superior precision and recall—validating its strong practicality for real-world deployment.

Keywords—Web Application Firewall, Blockchain, Machine Learning, Log Integrity, False Positive Reduction, Attack Detection

I. INTRODUCTION

As a critical component of web security defense systems, Web Application Firewall (WAF) generates massive volumes of security alert logs through real-time monitoring of HTTP/HTTPS traffic. However, existing logging systems face two major challenges: traditional centralized storage architectures lack robust anti-tampering mechanisms, making them susceptible to tampering by attackers who can erase their traces and complicate forensic evidence collection [1]; meanwhile, detection methods relying on static rules exhibit limited adaptability, resulting in persistently high false positive rates [2]. The prevalence of such false alarms significantly

impedes accurate identification and timely response to genuine threats. Consequently, ensuring log integrity, mitigating false alarms, and enabling in-depth analysis have become critical and ongoing research challenges in web security [3].

II. RELATED WORK

In the domain of WAF log storage, both centralized systems like Syslog [4] and distributed systems like Flume [5] fundamentally lack built-in tamper-proofing mechanisms. This inherent vulnerability undermines the evidential value of logs for post-incident forensic analysis. While blockchain technology inherently ensures data integrity, its direct application to voluminous WAF logs is hampered by performance and scalability constraints. Pioneering works by Taguchi et al. [6] and Pourmajidi et al. [7] effectively addressed this by proposing hybrid on-chain/off-chain models, where only cryptographic digests are stored on the blockchain. These approaches provide crucial references for the evidence preservation mechanism in this framework.

To implement the hybrid storage architecture, BML-WAF employs ChainMaker [8] as its underlying blockchain infrastructure. The platform's high-performance parallel consensus mechanism effectively overcomes throughput limitations in log storage operations, while its native support for customizable smart contracts facilitates automated evidence anchoring. These technical capabilities make ChainMaker particularly well-suited for implementing the tamper-evident storage layer, establishing a secure and efficient foundation for subsequent analytical components in the framework.

For WAF log analysis, reliance on static, rule-based methods consistently results in high false positive rates, overwhelming

security analysts [9]. While machine learning (ML) offers potential for improved accuracy, purely data-driven models often face their own challenges: they may require extensive, manual feature engineering and typically operate as “black boxes,” offering limited interpretability—a significant drawback in security contexts where justifying a decision is crucial [10]. Recent studies have begun to explore pathways to effectively integrate rules with ML. For instance, Arul et al. [9]. combined logistic regression with expert rules, offering valuable insights for the multi-layer feature fusion architecture introduced in this paper.

To systematically tackle the dual challenges of **secure storage** and **accurate analysis** of **WAF logs**, this paper proposes a unified framework, BML-WAF, which incorporates both on-chain/off-chain collaborative storage and multi-layer feature fusion analysis.

III. FRAMEWORK DESIGN

BML-WAF(Blockchain–Machine Learning–based WAF) is designed with a layered architecture for securing and analyzing WAF logs. The overall system, depicted in Fig. 1, is composed of five primary components: the Data Preprocessing Layer, the Blockchain-based Storage Layer, the Feature Extraction Layer, the Rule-based Calibration Layer, and the Heterogeneous Feature Fusion Layer.

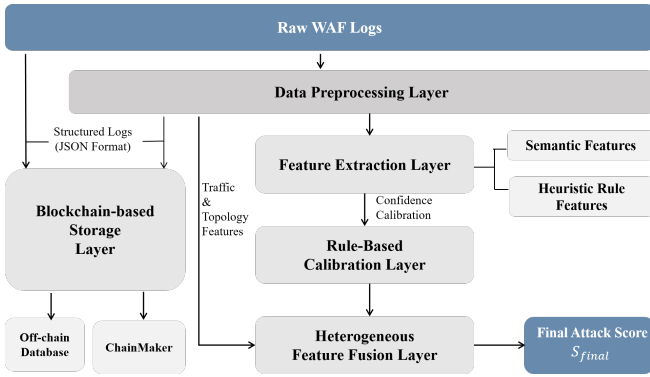


Fig. 1: Framework Overall Design

The BML-WAF framework follows a structured pipeline to ensure both the integrity and intelligibility of WAF logs. The process begins with **Raw WAF Logs** entering the **Data Preprocessing Layer**, executing the essential transformation of this raw data into a cleansed, structured and analysis-ready format. The processed logs then flow through two parallel paths: the **Feature Extraction Layer** converts log content into numerical semantic features, while the **Blockchain-Based Storage Layer** anchors log hashes immutably on the ChainMaker blockchain, with full logs stored securely in an **Off-chain Database**. The extracted features are subsequently refined by the **Rule-based Calibration Layer**, which incorporates Heuristic Rules to adjust the preliminary attack score. Finally, the **Heterogeneous Feature Fusion Layer** integrates semantic, rule-based, and other engineered features to produce

a **Final Attack Score**, enabling accurate and interpretable threat detection.

A. Data Preprocessing Layer

Raw WAF logs are highly heterogeneous in format and structure [11]. This layer is responsible for transforming raw, unstructured WAF logs into a clean, structured, and analysis-ready format. It involves three sequential steps:

1) *Data Cleansing*: The raw logs are first cleansed by removing irrelevant entries, eliminating duplicates, and imputing missing values to ensure data quality.

2) *Field Standardization*: Key alert fields (e.g., source IP, request URL, user agent) are parsed and extracted into a unified JSON structure. This standardization ensures consistency for subsequent feature extraction and blockchain hashing.

3) *Feature Enhancement*: Additional contextual and topological features are engineered from the raw data. These include traffic frequency from source IPs and categorical encodings for low-risk alert types, providing valuable discriminative signals for the downstream fusion layer. The output of this layer is a standardized log entry, primed for both secure storage and in-depth analysis.

B. Blockchain-based Storage Layer

To establish an anti-tamper log storage system, BML-WAF implements a blockchain-based storage layer leveraging the ChainMaker [8]. This layer accepts structured data from the Data Preprocessing Layer and operates on a hybrid on-chain/off-chain storage architecture, as illustrated in Fig. 2 .

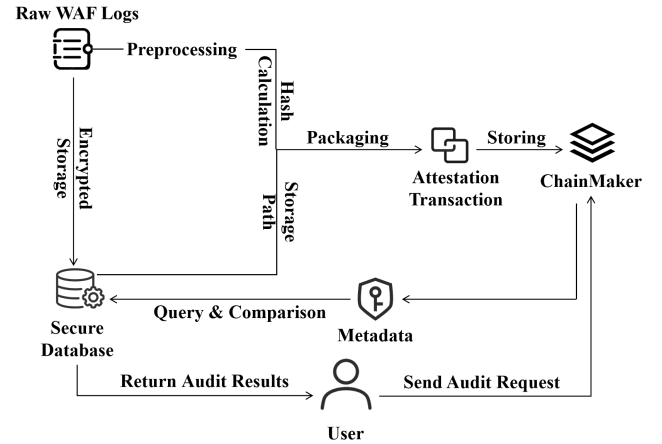


Fig. 2: Blockchain-Based Storage Layer Architecture

During the storage phase, processed logs are stored in the off-chain database, generating a corresponding access identifier. The SHA-256 hash algorithm is applied to compute a unique digest of the log data. This cryptographic hash, combined with relevant metadata, forms an evidence transaction that is submitted through a smart contract [8] to the ChainMaker blockchain. Following successful consensus validation, the transaction is immutably recorded on the distributed ledger.

During the auditing phase, the on-chain metadata is queried to retrieve the location of the corresponding off-chain log.

Integrity verification is performed by recomputing the hash value of the stored log and comparing it with the immutable record on the blockchain; any discrepancy immediately triggers a security alert. This mechanism not only ensures robust log integrity protection but also provides verifiable traceability for security auditing and compliance requirements.

C. Feature Extraction Layer

This layer converts preprocessed, structured logs into numerical feature representations for attack detection models, emphasizing textual semantic feature extraction and generating a baseline attack probability through logistic regression.

For textual semantic features, key fields (e.g., vulnerability type, attack type) in the standardized JSON are concatenated into a unified text sequence, as shown in (1), where f_n denotes the text content of the i -th field.

$$T = \text{concat}(f_1, f_2, \dots, f_n) \quad (1)$$

Subsequently, the unified text sequence T is mapped to a numerical feature representation by a TF-IDF vectorizer [10], emphasizing discriminative keywords for subsequent modeling. In this process, the term frequency (TF) of a term t in text T is defined in (2), where $\text{count}(t, T)$ denotes the number of occurrences of term t in text T .

$$\text{TF}(t, T) = \frac{\text{count}(t, T)}{\sum_{t' \in T} \text{count}(t', T)} \quad (2)$$

Inverse Document Frequency (IDF) is used to evaluate the global discriminative power of a term. It is calculated as shown in (3), where N is the total number of documents, N_t is the number of documents containing term t , and the addition of 1 is introduced to prevent the denominator from becoming zero.

$$\text{IDF}(t) = \ln \frac{N + 1}{N_t + 1} \quad (3)$$

The value of the TF-IDF characteristic is the product of TF and IDF, as shown in (4).

$$\text{TF-IDF}(t, T) = \text{TF}(t, T) \times \text{IDF}(t) \quad (4)$$

Input the extracted TF-IDF feature vector $X = (x_1, x_2, \dots, x_n)$ into the logistic regression(LR) model [12]. This model calculates the baseline attack probability S_{raw} through a linear weighted sum and a nonlinear mapping.

The LR model assigns weights w_i to each feature x_i and calculates the linear output z together with the bias term b , as shown in (5).

$$z = \mathbf{w} \cdot \mathbf{x} + b = \sum_{i=1}^n w_i x_i + b \quad (5)$$

In (6), the scalar z is mapped to the interval (0,1) via the sigmoid function $\sigma(z)$, yielding S_{raw} .

$$S_{raw} = \sigma(z) = \frac{1}{1 + e^{-z}} \quad (6)$$

The probability S_{raw} provides an initial assessment of attack behavior based on semantic features, which serves as the basis for subsequent rule refinement and multi-feature fusion.

D. Rule-Based Calibration Layer

Based on the attack probability S_{raw} from the feature extraction layer, this module performs intelligent calibration using a predefined security rule library. Dynamically adjusts the raw score to produce a refined score S_{adj} while generating interpretable rule-based features for subsequent multi-feature fusion.

The core score correction applies a weight adjustment strategy, as defined in (7), where w represents an expert-defined rule weight coefficient.

$$S_{adj} = S_{raw} \times w \quad (7)$$

Rule triggers [13] cover multiple security dimensions, including:

- Presence of defensive keywords in response content.
- User agent matching known scanning tool signatures.
- Original alerts classified as low-risk.
- Repetitive behavioral patterns from network entities within time windows.

Upon rule triggering, the weight adjustment mechanism is activated to suppress false positives and filter redundant alerts.

In parallel, the system records the semantic tags associated with the triggered rules and generates key Boolean features, including: response body characteristic f_{resp} , user agent characteristic f_{ua} , and risk level characteristic f_{low} .

The rule refinement layer ultimately outputs the calibrated confidence score S_{adj} along with a set of interpretable rule-based features. These outputs collectively serve as inputs to the subsequent multi-feature fusion layer, facilitating both precise and explainable threat detection.

E. Heterogeneous Feature Fusion Layer

As the final stage of the detection pipeline, this layer integrates multi-source features—including the calibrated probability S_{adj} , Boolean rule-based features from the rule refinement layer, and traffic-topology features from the preprocessing layer—to compute a comprehensive attack probability score.

To adapt the baseline probability for linear modeling, S_{adj} is transformed via a logit function, mapping it to the entire real number space and forming the base feature f_{base} , as defined in (8).

$$f_{base} = \begin{cases} \ln \left(\frac{S_{adj}}{1 - S_{adj}} \right), & \text{if use_logit} = \text{True} \\ S_{adj}, & \text{if use_logit} = \text{False} \end{cases} \quad (8)$$

Rule features f_{rule} are represented by feature vectors composed of three key Boolean variables, as defined in (9).

$$f_{rule} = [f_{resp}, f_{ua}, f_{low}] \quad (9)$$

Each component corresponds to response body whitelist matching, scanner user-agent identification, and low-risk/low-confidence determination, respectively. A value of 1 indicates a rule match, while 0 indicates no match.

In (10), the complete input feature vector $\mathbf{F}$ is constructed by combining the base features, rule features, and the traffic and topology features $f_{traffic}$ from the preprocessing layer.

$$\mathbf{F} = [f_{base}, f_{rule}, f_{traffic}] \quad (10)$$

The fusion process leverages a logistic regression model to compute the final attack probability, applying the linear weighting and nonlinear transformation specified in (11):

$$S_{final} = \sigma(\mathbf{w}^T \mathbf{F} + b) \quad (11)$$

where $\sigma(x) = \frac{1}{1+e^{-x}}$ is the Sigmoid function, $\mathbf{w}$ is the weight vector, and b is the bias term.

The model training utilizes the cross-entropy loss as the optimization objective, as shown in (12).

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (12)$$

where y_i denotes the ground-truth label, $\hat{y}_i$ represents the predicted probability, and N is the total number of samples. To mitigate class imbalance, a class weight of 2.0 is assigned to the attack category during training.

The threshold selection mechanism [14] is based on the precision-recall curve. The optimal threshold θ^* is determined according to the following criterion, as shown in (13):

$$\theta^* = \begin{cases} \arg \max_{\theta} P(\theta), & \text{if } R(\theta) \geq 0.99 \\ \arg \max_{\theta} F_1(\theta), & \text{otherwise} \end{cases} \quad (13)$$

where $P(\theta)$ and $R(\theta)$ denote the precision and recall at threshold θ respectively, and $F_1(\theta)$ is the corresponding F_1 -score.

The fusion module outputs the final attack probability score S_{final} , a list of salient features that contribute significantly to the decision, and the labels of triggered rules. This design enhances interpretability by revealing influential features and rule-based reasoning, thereby achieving a unified framework that balances detection accuracy and explanatory transparency.

IV. RESULTS AND DISCUSSION

A. Dataset

The experiment utilized the publicly available cybersecurity alarm dataset [15], which covers various typical attack types such as SQL injection, command execution, information leakage, and scanning activities. The core information of the dataset is summarized in TABLE I.

TABLE I: Core Information of the Dataset

Attribute	Description
Total Samples	65,134
Attack Samples	291 (0.45%)
False Alarm Samples	64,843 (99.55%)
Data Format	Instruction-Input-Output
Input Format	JSON-formatted Alert Logs
Output Labels	"Attack" / "False Alarm"

The dataset exhibits a severe class imbalance, with false positives accounting for 99.55% of the samples and genuine

attacks making up only 0.45%. This distribution accurately reflects real-world network environments, which are dominated by false alarms [1]. Therefore, it provides an effective basis for validating model generalization and robustness in authentic scenarios.

B. Evaluation Metrics

The core metrics and calculation formulas used to evaluate framework performance are shown in TABLE II.

TABLE II: Evaluation Metrics and Formulas

Evaluation Metric	Formula
False Positive Rate (FPR)	$FPR = \frac{FP}{FP+TN}$
Precision	$Precision = \frac{TP}{TP+FP}$
Average Precision (AP)	$AP = \sum_n (R_n - R_{n-1}) P_n$
Recall	$Recall = \frac{TP}{TP+FN}$
F1-Score	$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$

C. Tamper Evidence Verification

To comprehensively evaluate the framework's capability in safeguarding log integrity, the dataset was partitioned into four distinct test cohorts, each designed to simulate a specific type of tampering operation: content replacement, field deletion, timestamp manipulation, and metadata alteration. From the original collection of 65,134 log instances, 1,000 records were randomly sampled per cohort for targeted tampering. The complete verification workflow is depicted in Fig. 3.

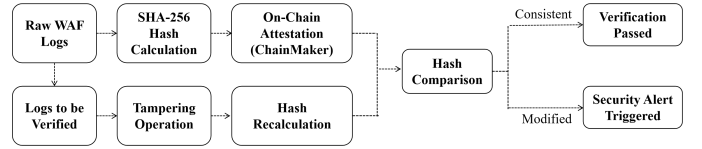


Fig. 3: Tamper-Resistance Verification Workflow

Evaluation results indicate that the blockchain-based hash attestation mechanism exhibits excellent detection performance under various tampering scenarios. As summarized in Fig. 4, the framework achieved an average detection accuracy of **99.8%** over the four defined operation types. Perfect accuracy **100%** was observed for content replacement and field deletion, with timestamp tampering and metadata modification also being identified at high accuracies of **99.7%** and **99.6%**, respectively.

The integrity verification process, in which the system computes the SHA-256 hash of each log entry in real time and compares it precisely against the original hash stored on the blockchain, demonstrated consistently high effectiveness. Although a minimal number of timestamp and metadata tampering cases were not immediately detected due to factors such as system clock synchronization and network latency, all were successfully identified in subsequent batch verification. Overall, any log tampering invariably resulted in a hash mismatch, triggering the security alert mechanism and thereby validating the practical reliability of the proposed framework for protecting WAF log integrity.

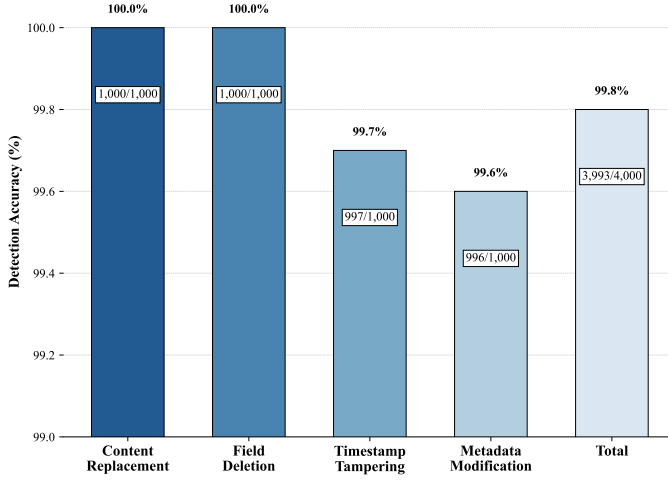


Fig. 4: Detection Accuracy for Different Tampering Types

D. False Positive Reduction

To systematically evaluate the framework's performance in false alarm suppression, three key metrics were adopted: FPR, Precision, and AP. Experimental results demonstrate the framework's outstanding effectiveness in this regard, achieving an FPR of **0.0005**, a Precision of **0.9016**, and an AP of **0.9749**. These results indicate that, even on a highly class-imbalanced dataset, the framework successfully suppresses the vast majority of false alerts while maintaining high discriminative accuracy towards true attack instances.

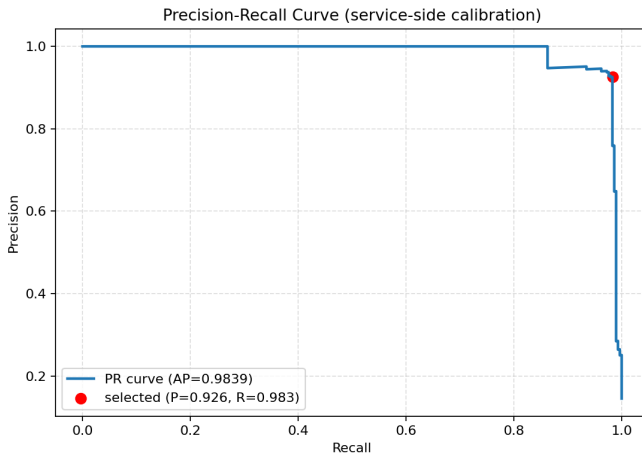


Fig. 5: Precision-Recall Curve

As illustrated in the precision-recall curve of Fig. 5, following server-side calibration, the model achieves a precision of **0.926** and a recall of **0.983** at the operating threshold, with the average precision reaching **0.9839**. This outcome demonstrates that the framework maintains high decision confidence while meeting stringent recall requirements, thereby effectively balancing the critical trade-off between false alarm suppression and attack detection.

E. Attack Detection Performance

The evaluation of attack detection performance focuses on the model's capability to identify genuine threats, with Recall, F1-Score, and AP selected as the core metrics. Experimental results show that the proposed framework achieves a recall of **0.9483** and an F1-Score of **0.9244** on the test set, demonstrating its ability to effectively capture the vast majority of attack activities while maintaining a strong balance between precision and recall. Furthermore, an AP score of **0.9749** confirms the model's robustness in high-precision regions.

As shown in the precision-recall curve in Figure 5, the overall curve lies close to the top-right corner, indicating that the model sustains high precision across varying recall levels. This result validates the framework's capacity to simultaneously achieve high attack detection rates and low false alarms in real-world scenarios.

F. Comprehensive Performance Evaluation

The proposed framework's overall performance for WAF log analysis was evaluated through systematic comparison with Random Forest, a widely adopted and robust baseline. Random Forest [16] was selected based on its demonstrated effectiveness in security detection tasks, capability to handle high-dimensional features [17] while capturing nonlinear relationships, and inherent interpretability [18] through feature importance assessment. The comparative evaluation was conducted under identical experimental conditions, with results detailed in Fig. 6.

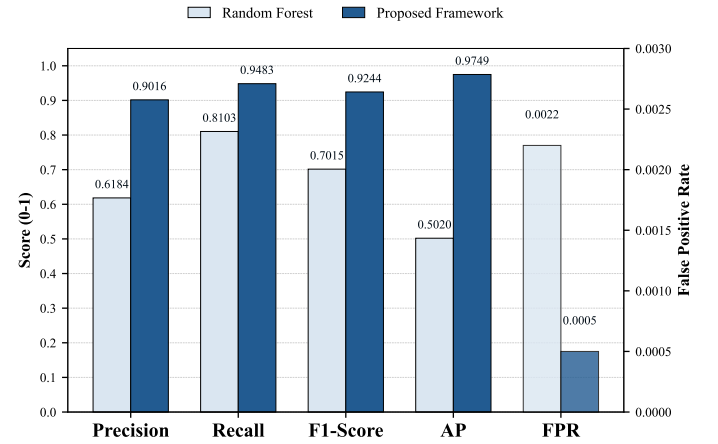


Fig. 6: Key Metrics Comparison

1) False Alarm Suppression: The proposed framework achieves a substantially lower false positive rate of 0.0005 compared to the Random Forest baseline of 0.0022, representing a **77.27%** reduction. This improvement stems from the rule-based correction layer, which incorporates explicit domain knowledge to filter false alarm patterns at the source—a capability lacking in the purely data-driven Random Forest approach. While Random Forest relies solely on statistical correlations learned from training data, the proposed framework

actively eliminates known benign patterns through security-specific rules, thus demonstrating superior performance in minimizing erroneous alerts.

2) *Detection Accuracy and Robustness*: The proposed framework demonstrates superior detection performance with a precision of 0.9016 and F1-score of 0.9244, exceeding the Random Forest baseline by **3.49%** and **2.51%** respectively. This improvement is attributed to the multi-layer feature fusion architecture that systematically integrates complementary feature types, providing more discriminative capability than Random Forest's implicit feature interactions. The framework's explicit feature design enables more reliable decision boundaries, particularly under the challenging class-imbalanced conditions characteristic of security log data.

3) *Attack Detection Capability*: The proposed framework maintains a high recall rate of 0.9483 while achieving an exceptionally low false positive rate, outperforming the Random Forest baseline by **1.41%** in recall. This demonstrates the framework's ability to effectively identify genuine threats without compromising on false alarm suppression—a critical balance that Random Forest fails to achieve due to its inherent limitation in incorporating domain-specific knowledge for precise pattern discrimination. The results confirm the framework's practical advantage in real-world deployment scenarios where both high detection coverage and operational reliability are essential.

In summary, the integration of semantic analysis, probabilistic modeling, rule-based correction, and multi-feature fusion within the proposed framework demonstrates its superior performance over Random Forest across all critical metrics for WAF log analysis.

V. CONCLUSION

This paper proposes BML-WAF, an integrated framework for WAF log management that synergizes blockchain-based integrity protection with machine learning-based analysis. The framework establishes a hybrid on-chain/off-chain storage architecture to ensure log immutability, coupled with a three-layer analytical structure incorporating feature extraction, rule-based correction, and multi-feature fusion for comprehensive threat detection.

Experimental evaluation demonstrates significant performance improvements over the Random Forest baseline across all key metrics. The blockchain-based verification mechanism achieves **99.8%** tampering detection accuracy. In false alarm suppression, the framework reduces the false positive rate by **77.27%** compared to the baseline. For detection capability, it achieves a **3.49%** improvement in precision, **1.41%** higher recall, and **2.51%** better F1-score, while maintaining a **2.17%** advantage in average precision - all accomplished under extreme class imbalance conditions.

Future research directions include implementing attribute-based encryption for fine-grained access control and developing cross-domain collaboration mechanisms to enable secure log sharing and collaborative analysis among multiple organi-

zations, thereby further enhancing the framework's applicability in complex operational environments.

REFERENCES

- [1] I. Ray, K. Belyaev, M. Strizhov, D. Mulamba, and M. Rajaram, "Secure logging as a service—delegating log management to the cloud," *IEEE Systems Journal*, vol. 7, no. 2, pp. 323–334, 2013.
- [2] M. Astekin, H. Zengin, and H. Sözer, "Evaluation of distributed machine learning algorithms for anomaly detection from large-scale system logs: A case study," in *2018 IEEE International Conference on Big Data (Big Data)*, pp. 2071–2077, 2018.
- [3] J. Harish Kumar and J. Godwin Ponsam, "Securing web application using Web Application Firewall (WAF) and machine learning," in *2023 First International Conference on Advances in Electrical, Electronics and Computational Intelligence (ICAECCI)*, pp. 1–8, Oct 2023.
- [4] S. Zhang, Y. Liu, W. Meng, J. Bu, S. Yang, Y. Sun, D. Pei, J. Xu, Y. Zhang, L. Song, and M. Zhang, "Efficient and robust syslog parsing for network devices in datacenter networks," *IEEE Access*, vol. 8, pp. 30245–30261, 2020.
- [5] W. Peng, Y. Li, B. Li, and X. Zhu, "An analysis platform of road traffic management system log data based on distributed storage and parallel computing techniques," in *2016 IEEE International Conferences on Big Data and Cloud Computing (BDCloud), Social Computing and Networking (SocialCom), Sustainable Computing and Communications (SustainCom) (BDCloud-SocialCom-SustainCom)*, pp. 585–589, 2016.
- [6] Y. Taguchi, A. Kanai, and S. Tanimo, "A distributed log management method using a blockchain scheme," in *2020 IEEE International Conference on Consumer Electronics (ICCE)*, pp. 1–3, 2020.
- [7] W. Pourmajidi and A. Miransky, "Logchain: Blockchain-assisted log storage," in *2018 IEEE 11th International Conference on Cloud Computing (CLOUD)*, pp. 978–982, 2018.
- [8] "Chainmaker official website." <https://chainmaker.org.cn/home>. Accessed: Nov. 21, 2025.
- [9] A. Kumar Natarajan, M. Gouse Galety, S. Noekhah, R. Soundararajan, S. Xurshed, and Z. Xasan, "Enhancing cybersecurity in smart grid systems through advanced log file analysis with machine and deep learning techniques," in *2024 Third International Conference on Sustainable Mobility Applications, Renewables and Technology (SMART)*, pp. 1–10, 2024.
- [10] H. Zhang, S. Li, X. Wu, W. Han, and L. Wang, "An android malware detection approach using multi-feature fusion and tf-idf algorithm," in *2021 IEEE Sixth International Conference on Data Science in Cyberspace (DSC)*, pp. 629–634, 2021.
- [11] Z. Qu, X. Ling, T. Wang, X. Chen, S. Ji, and C. Wu, "Advsqli: Generating adversarial sql injections against real-world waf-as-a-service," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 2623–2638, 2024.
- [12] M. Salari, R. Ghanbari, and K. Ghorbani-Moghadam, "Least squares method for solving fuzzy lr interval algebraic linear systems," *Fuzzy Information and Engineering*, vol. 14, no. 3, pp. 349–359, 2022.
- [13] M. Vukadinovic, B. Reiterer, M. Rathmair, and C. G. Schuetz, "Anomaly detection in robot applications: Comparison of rule-based and machine learning methods," in *2024 9th International Conference on Control, Robotics and Cybernetics (CRC)*, pp. 1–5, 2024.
- [14] J. Long, J. Zhang, N. Xiang, J. Zhang, and D. Wang, "An iterative maximum entropy thresholding algorithm," in *2016 International Conference on Cyberworlds (CW)*, pp. 171–174, 2016.
- [15] tiangler, "cybersecurity_alarm_analysis." Hugging Face Datasets, 2025. Dataset available at https://huggingface.co/datasets/tiangler/cybersecurity_alarm_analysis (Accessed: Nov. 22, 2025).
- [16] M. Choubisa, R. Doshi, N. Khatri, and K. Kant Hiran, "A simple and robust approach of random forest for intrusion detection system in cyber security," in *2022 International Conference on IoT and Blockchain Technology (ICIBT)*, pp. 1–5, 2022.
- [17] V. Kesarwani and R. E., "Advanced detection of malicious urls using machine learning : A comparative analysis of svm, random forest, and logistic regression," in *2024 1st International Conference on Advances in Computing, Communication and Networking (ICAC2N)*, pp. 228–233, 2024.
- [18] E. P. Ghani, A. Sofwan, and M. Somantri, "Ai-driven network security: Detecting and mitigating ddos, malware, and backdoor attacks with isolation and random forest algorithm," in *2025 International Conference on Smart Computing, IoT and Machine Learning (SIML)*, pp. 1–6, 2025.